

Introduction

Scientific Machine Learning and Simulation Intelligence

Who am I?

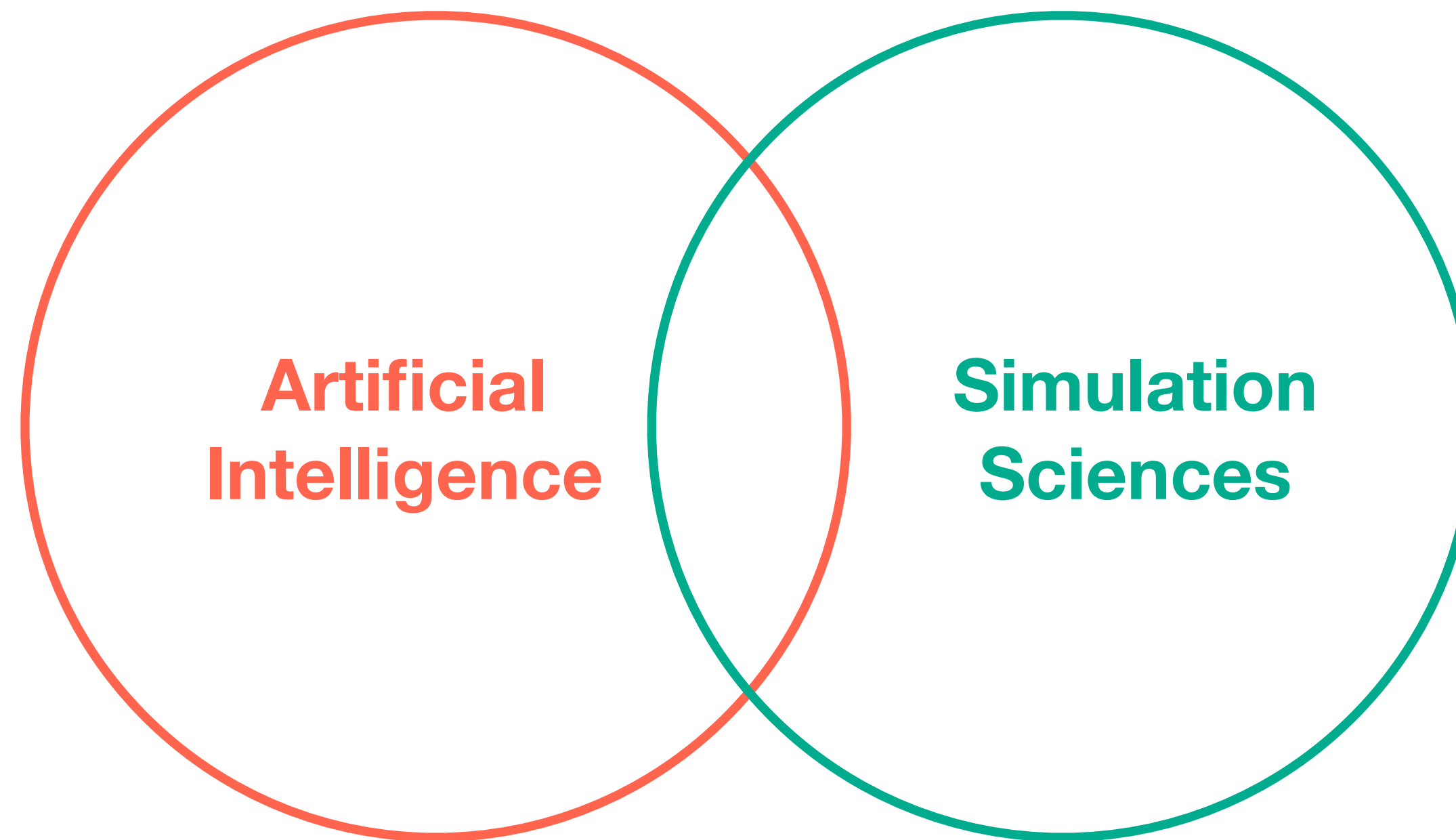
Francesca Cairolì

E-mail: francesca.cairolì@units.it

Office: Via Giovanni e Demetrio Economo, 12/3, Room 4.15

Simulation intelligence

Intersection of AI and simulation sciences



Simulation lets us study complex systems through mathematical and computational models, while **Machine Learning** allows us to learn patterns and approximate complex relationships from data.

The interesting part is how these two areas can support each other: ML can accelerate and improve simulations, while scientific knowledge can make ML models more reliable and physically meaningful.

Simulation

Simulation is one of the most important tools available in science and engineering.

The basic idea is simple: instead of experimenting directly with the real system, we construct a mathematical or computational model and use it to study how the system behaves under different conditions.

This is especially useful when real experiments are expensive, dangerous, slow, or simply impossible.

A simulator allows us to explore not only situations that we have already observed, but also **hypothetical scenarios**. We can change parameters, initial conditions, external inputs, or even consider extreme events.

Simulation is widely used in fields such as climate science, biology, engineering, chemistry, medicine, and many others.

Problems and Limitations of Simulation

- The first one is **computational cost**. As we try to model systems with greater detail, higher spatial resolution, more physical processes, or longer time horizons, simulations can become extremely expensive.

And the problem is that scientific tasks rarely require just one simulator run. Optimisation, sensitivity analysis, uncertainty quantification, or inverse problems may require thousands or even millions of evaluations.

- This often forces us to introduce **simplifying assumptions**, reducing the realism of the model.
- Another limitation is that simulators are inherently **incomplete**: they reproduce only the mechanisms that have explicitly been included in the model.
- Finally, many traditional simulation codes are difficult to combine with modern machine-learning techniques. They may contain discrete operations, iterative numerical solvers, or legacy code that is **not differentiable**.

So even if we have a very good simulator, it may still be expensive, difficult to calibrate, difficult to integrate with data, and difficult to optimise.

Progress in AI and ML

At the same time, Machine Learning has evolved beyond classical tasks such as classification or pattern recognition.

Modern ML gives us tools to **incorporate prior knowledge into learning**, **approximate expensive functions**, **generate synthetic data**, **quantify uncertainty**, and **solve complex optimisation problems**.

Simulation and ML have complementary strengths.

A simulator provides us with **mechanistic knowledge**: equations, constraints, causal structure, and physical interpretation.

Machine learning provides us with **flexibility**, computational efficiency, and the ability to learn unknown relationships directly from data.

The goal is therefore not necessarily to replace simulation with ML, but to understand how these two approaches can work together.

Scientific ML (SciML)

Traditional deep learning has achieved remarkable results in areas where very large datasets are available, such as computer vision or natural language processing.

But many scientific problems are very different. Experimental data may be scarce because **experiments** are **expensive, difficult, or time-consuming**.

At the same time, in scientific problems we often possess something that is missing in generic machine-learning tasks: **domain knowledge**.

We may know differential equations, conservation laws, symmetries, interactions between variables, or qualitative properties of the system.

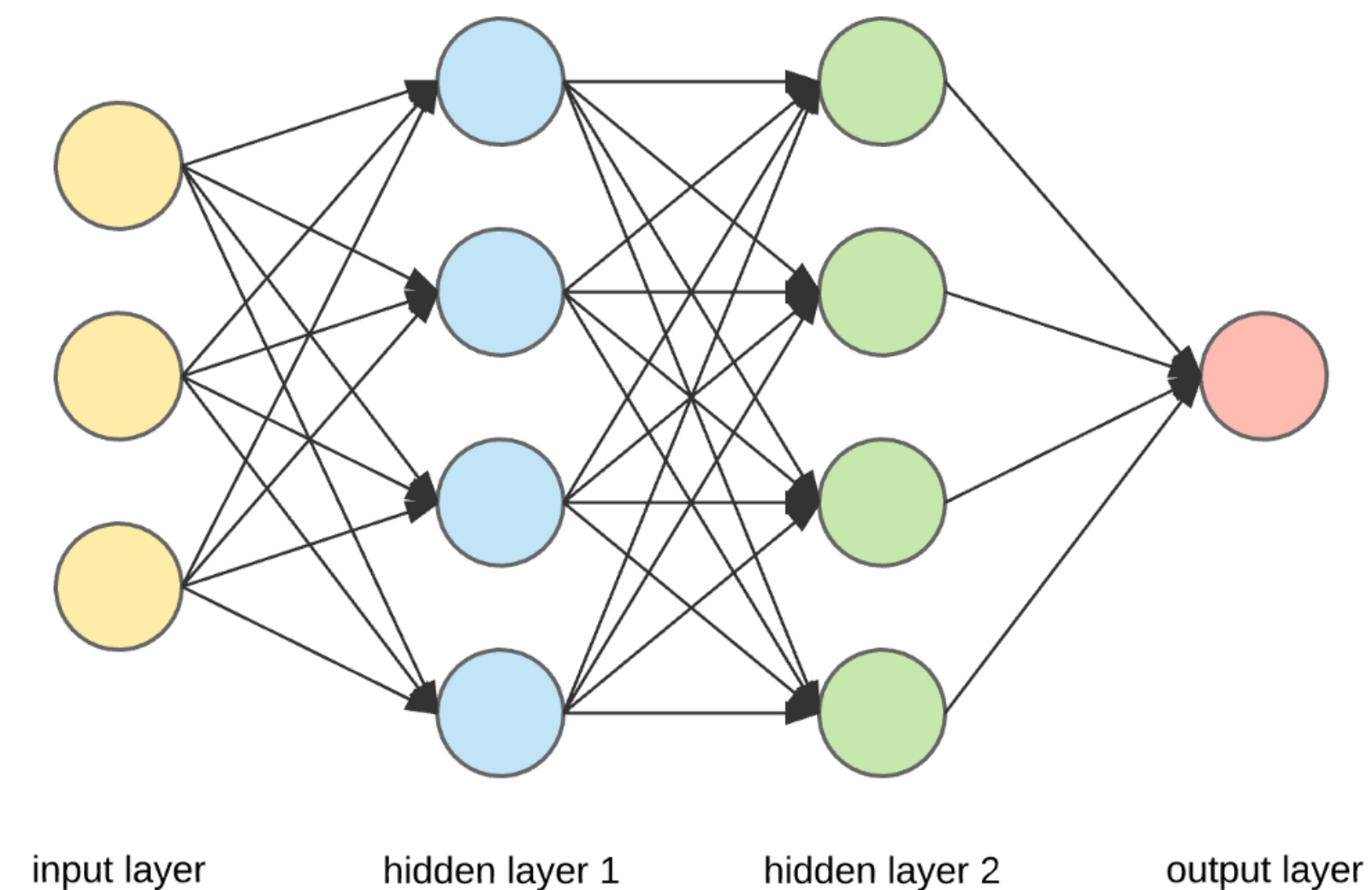
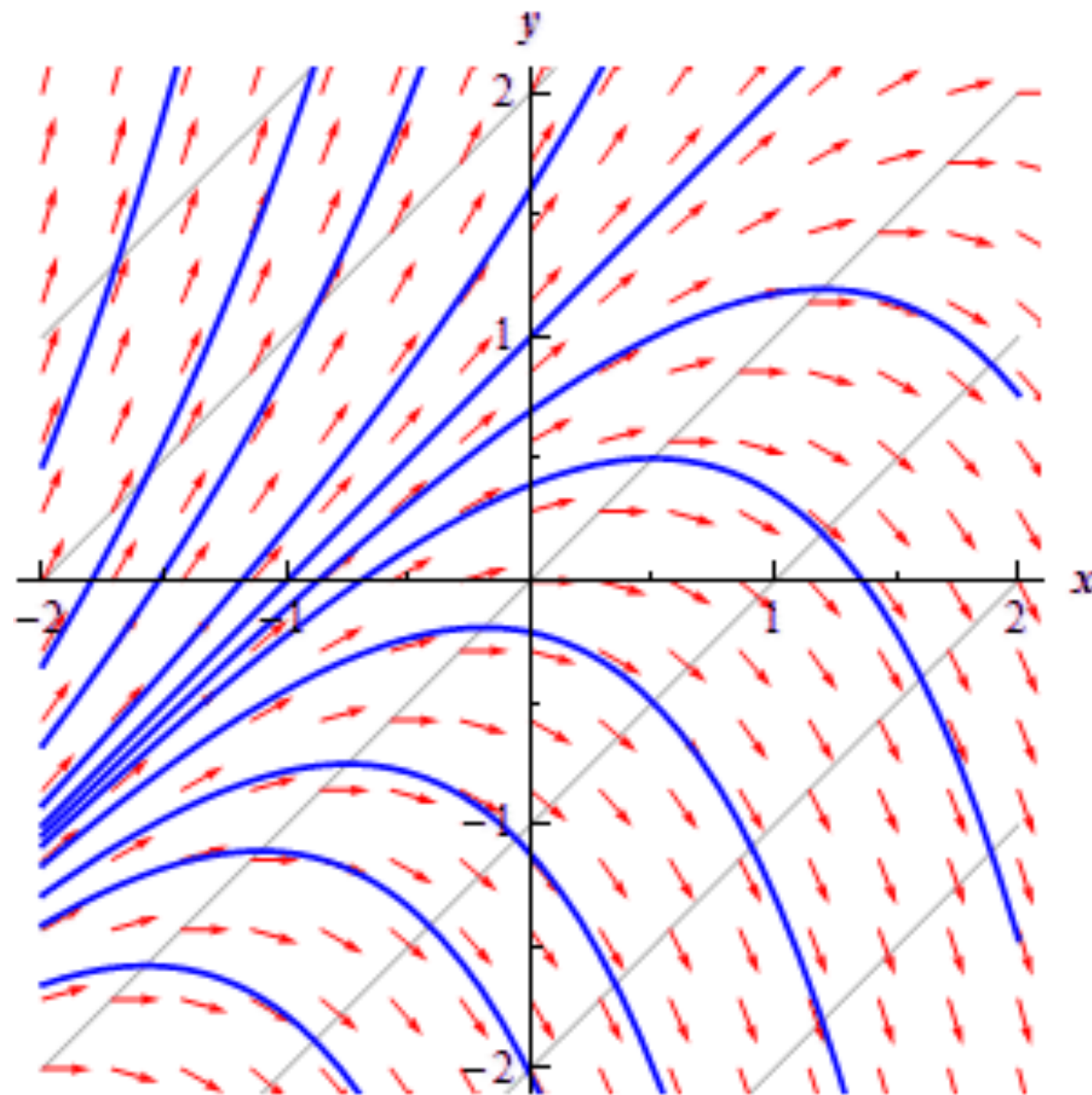
Scientific Machine Learning tries to exploit this information rather than ignoring it.

So, in very simple terms, SciML combines **data-driven learning** with **scientific and mechanistic knowledge**.

Scientific ML (SciML)

Integrate machine learning derived idea into traditional engineering-related methods in order to utilize domain knowledge and known physical information into the learning process.

Merging differential equations with machine learning.



SciML objectives

Development of SciML can help address imperfections of mechanistic models, help to build more resource efficient mechanistic models, and can help to discover new knowledge.

Different **objectives to develop and use** SciML from the perspective of improving computational techniques, stemming from the limitations of both process-based modelling and machine learning:

1. **improve prediction performance**, i.e. better matching between predictions and observations
2. **improve sample efficiency**, *i.e.* reducing the number of observations needed for adequate performance or to reduce solution search space
3. **improving the interpretability** of machine learning outcomes.

Applications of SciML

Introducing scientific knowledge can help to get a better understanding of processes. Range of objectives that help to improve modelling and process understanding:

1. **Improve over physical models:** combining physics-based models with ML-models;
2. **Downscaling:** use machine learning to simulate at a finer resolution.
3. **Parameterisation:** replacing complex sub-processes with machine-learning models instead of calibrated constants.
4. **Reduced-order models:** simplified, cheaper models whose accuracy and efficiency can be improved with physics-informed machine learning.
5. **Forward solving PDEs:** ML can provide faster alternatives to numerical PDE solvers, especially when combined with physical constraints.

Applications of SciML

6. **Inverse modelling:** ML surrogates enable fast, high-dimensional inverse modelling, while physics constraints improve robustness and data efficiency.
7. **Discovering governing equations:** ML can infer governing equations from data, e.g. using symbolic or sparse regression.
8. **Data generation:** ML can generate synthetic scientific data faster and with greater diversity than traditional simulations.
9. **Uncertainty quantification:** ML surrogates can reduce computational costs, while physics constraints improve uncertainty estimates and physical consistency.

SciML Methods

Different techniques can be used:

1. **Physics-guided loss:** add physics-based penalties to the loss function to discourage non-physical solutions and improve generalisation, as in **PINNs**.

2. **Physics-guided initialisation:** use physical knowledge to initialise or pre-train models, reducing training time and data requirements through techniques such as **transfer** or **self-supervised learning**.

Ex: Pre-training based on synthetic data generated by physics model. Then finetuning on few real data.

3. **Physics-guided architecture:** embed physical knowledge directly into the neural network structure to improve interpretability and physical consistency.

Ex: Graph Neural Network

4. **Hybrid physics-ML models:** combine physics-based models with ML components, for example to replace sub-models or correct residual errors.

Ex: $y_{true} \approx y_{physics} + f_{\theta}(x)$
neural residual

SciML Methods

1. **Physics-guided loss function:**

- i) *Requirement*: known physical relationship (e.g. PDE)
- ii) *Possible benefits*: physical consistency, improved generalisation, reduced observations required, improved accuracy

2. **Physics-guided initialisation:**

- i) *Requirement*: Synthetic data from mechanistic model for training
- ii) *Possible benefits*: Reduced observation required, improved accuracy

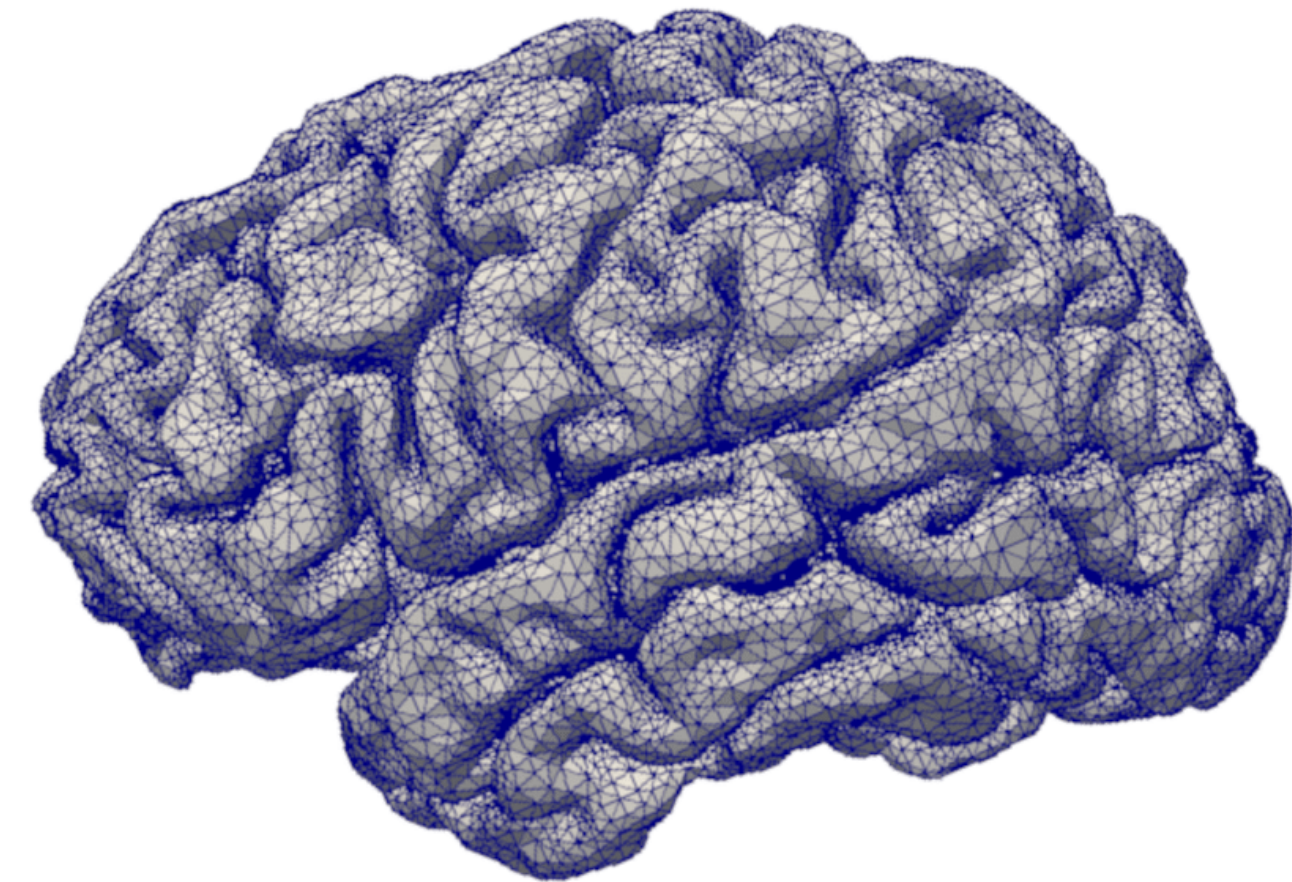
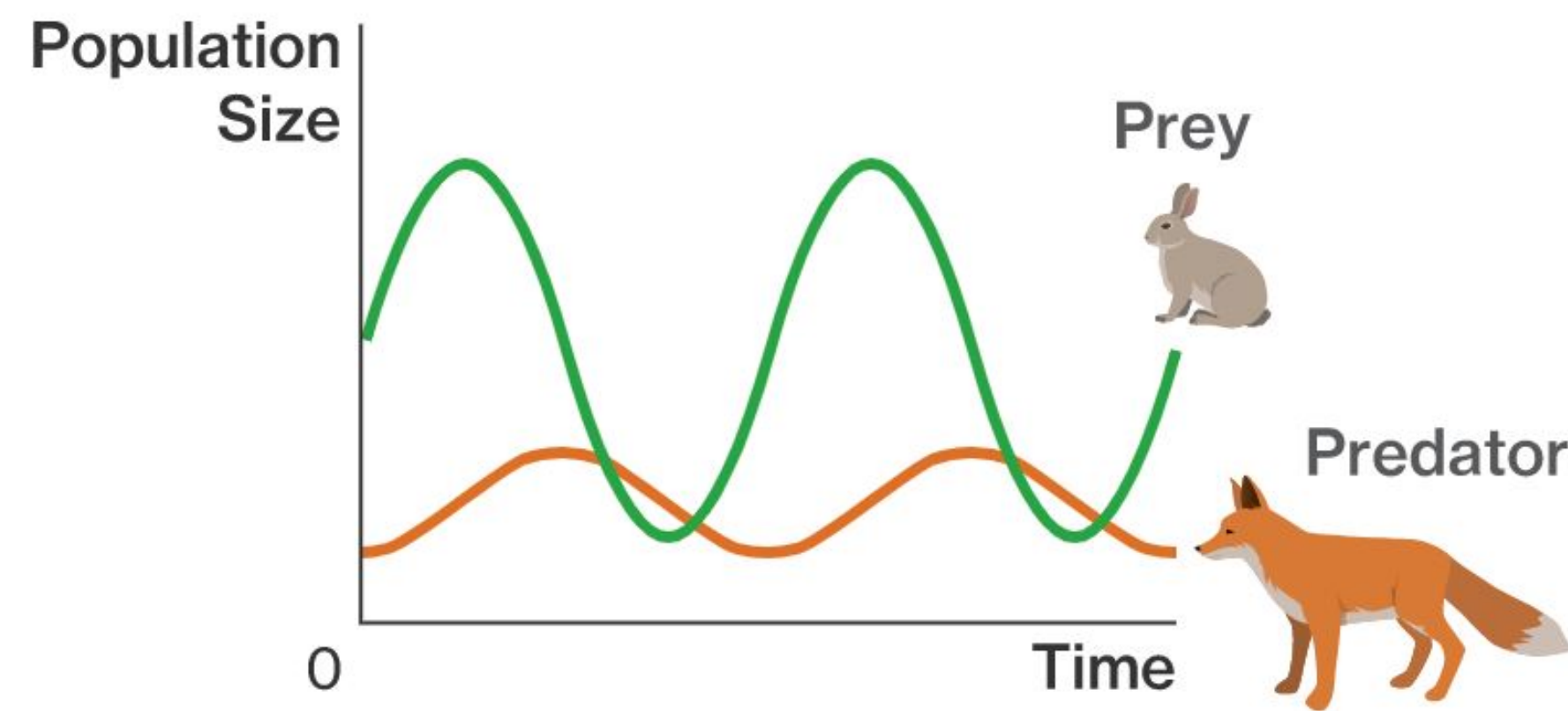
3. **Physics-guided design of architecture:**

- i) *Requirement*: Intermediate physical variables/processes, or hard constraints, or task interrelationships, or informed prior distributions
- ii) *Possible benefits*: Interpretability, physical consistency, improved generalisation, reduced solution search space, improved accuracy

4. **Hybrid physics-ML models:**

- i) *Requirement*: operational mechanistic model during run time
- ii) *Possible benefits*: improved accuracy

Simulators and surrogate modeling



Simulators can range, in **complexity**, from extremely simple models that have a conceptual or even pedagogical flavor (e.g. the Lotka-Volterra equations describing predator-prey interactions in ecological systems and also economic theories) to extremely detailed and expensive simulations implemented in supercomputers, e.g. whole-brain simulations.

Surrogates and Emulators

A surrogate model is an **approximation** that mimics the behaviour of an expensive computation or process.

Example: the design of an aircraft requires computationally intensive simulations with numerical optimisations that may take days to complete, making design space exploration, sensitivity analysis and inverse modelling **infeasible**.

A computationally efficient surrogate model can be trained to represent the system, **learning a mapping from simulator inputs to outputs**.

statistical emulation  replace a simulator with a machine learning model
so that retrieving the simulator outputs is more efficient

Data-driven surrogate models

Surrogate models are constructed using a data-driven, bottom-up approach.

The exact, inner working of the simulation code is not assumed to be known, relying solely on the input-output behaviour. A model is constructed based on modeling the response of the simulator to a limited number of intelligently chosen data points. This approach is also known as behavioural or black-box modeling.

Goals

Generate a surrogate as accurate as possible, using as few simulation evaluations as possible.

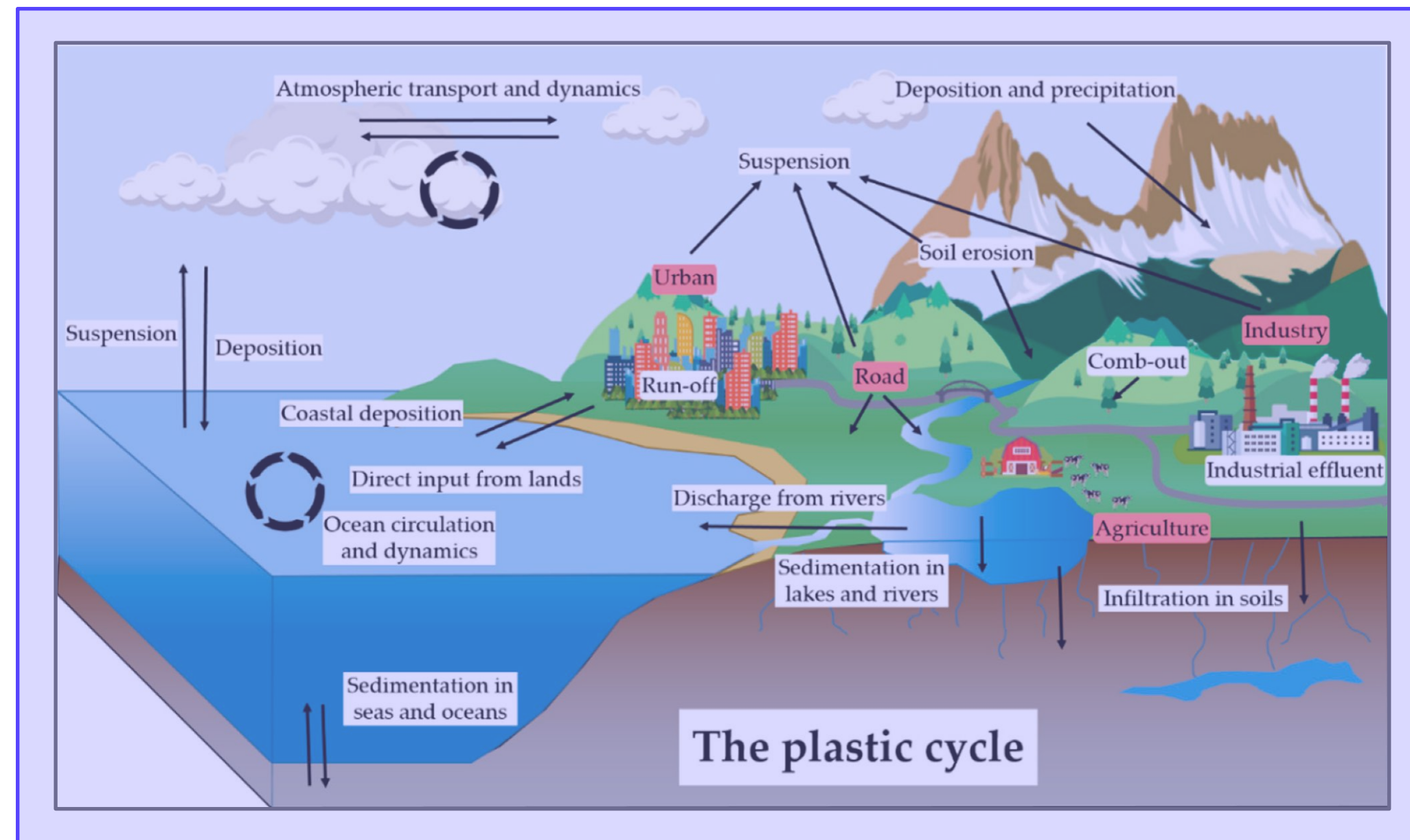
Three major steps which may be interleaved iteratively:

- Sample selection (a.k.a. optimal experimental design or active learning)
- Construction of the surrogate model and optimisation of the model parameters
- Appraisal of the accuracy of the surrogate model.

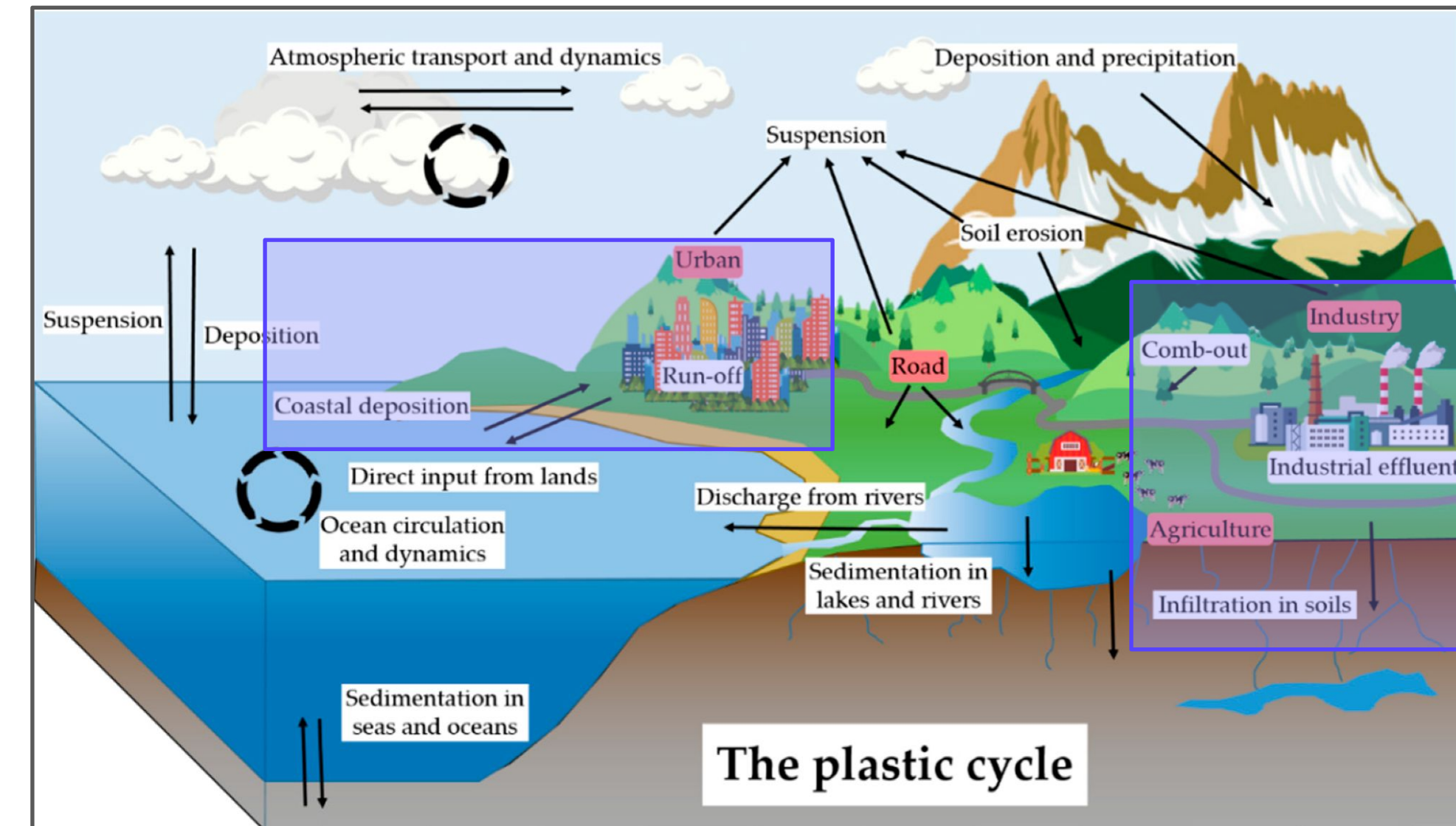
The accuracy of the surrogate depends on the number and location of samples in the design space.

Various design of experiments techniques cater to different sources of errors, in particular, errors due to noise in the data or errors due to an improper surrogate model.

Large ML surrogate model
represents entire simulator



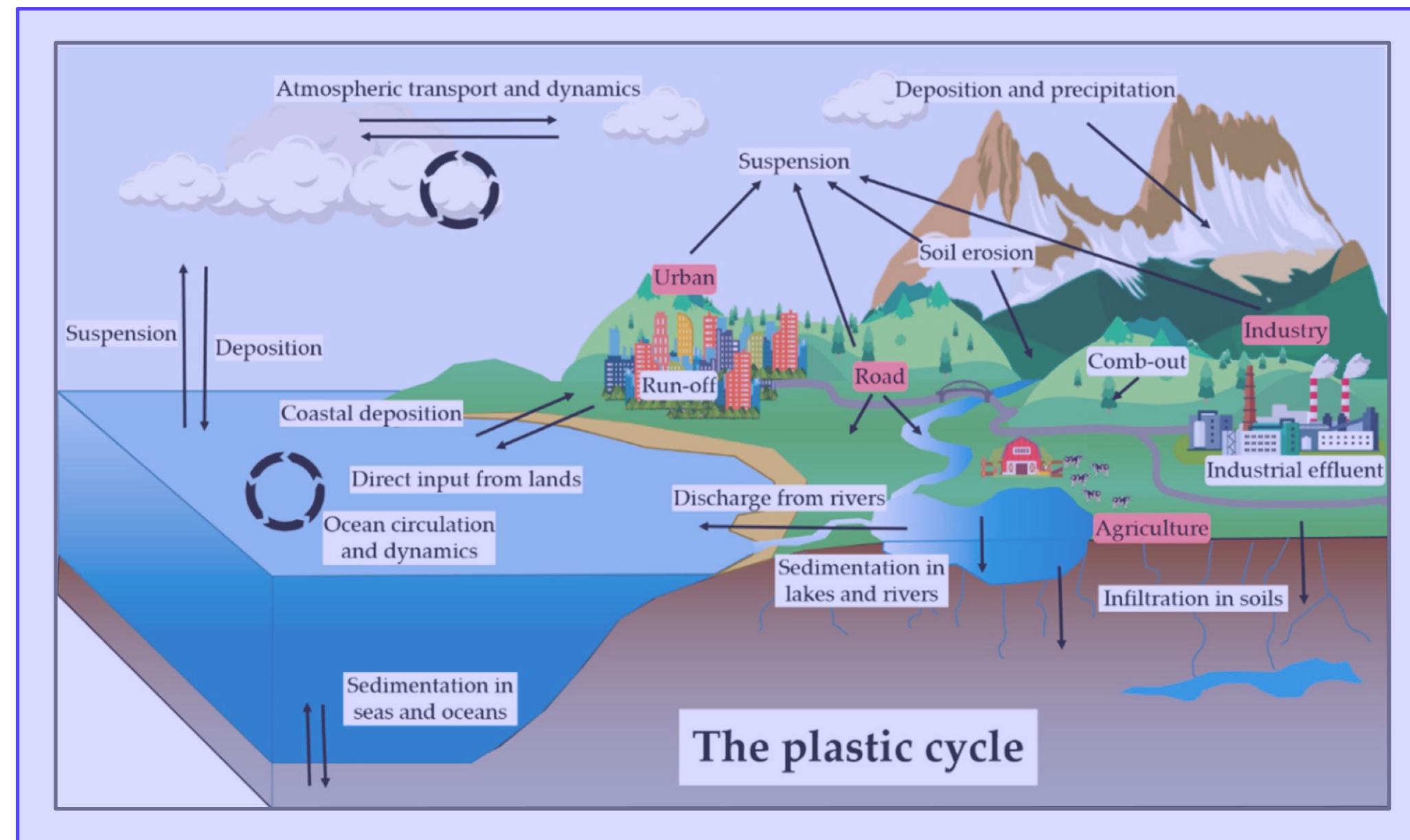
Small ML surrogates
represent specific simulator subsets
(unknown components, computational bottlenecks)



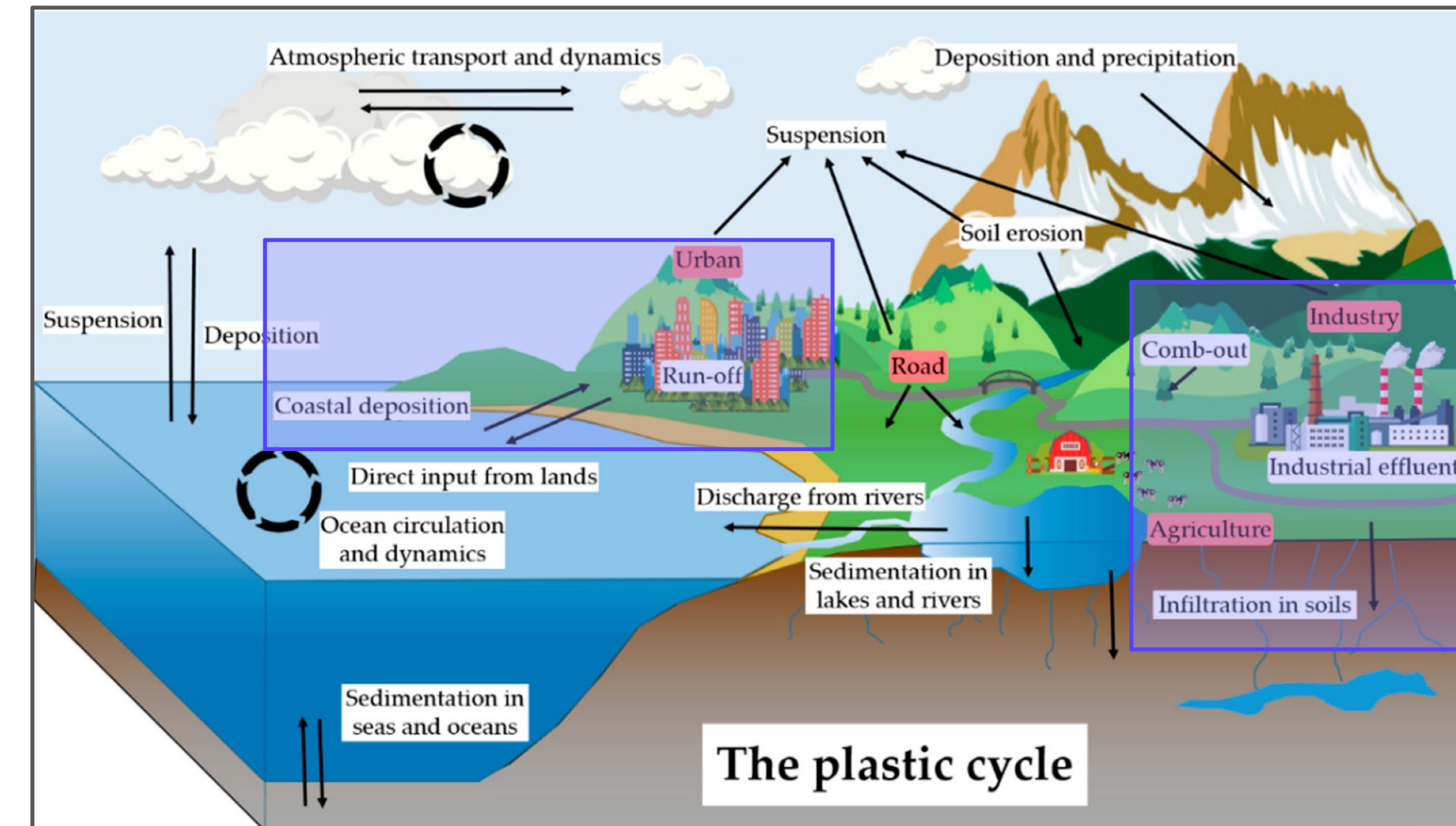
Emulator can encapsulate either the **entire simulator** or a **specific part** of it.

In the case of a **global surrogate**, the training is relatively straightforward. The trained surrogate can be used towards inverse problem solving. The simulator is now a black-box, no interpretability and we cannot utilize the mechanistic components (i.e. differential equations of the simulator) for scientific analyses.

Large ML surrogate model
represents entire simulator



Small ML surrogates
represent specific simulator subsets
(unknown components, computational bottlenecks)



Emulator can encapsulate either the **entire simulator** or a **specific part** of it.

In the case of the **partial surrogate**, we have several advantages: the surrogate's number of parameters is reduced and thus the emulator is more stable, the simulator retains all structural knowledge and the ability to run numerical analysis. Yet the main challenge is that backpropagation of arbitrary scientific simulators is required – thus the significance of the differentiable programming motif.

Multi-Physics

Complex real-world problems require **solutions that span a multitude of physical phenomena**, which often can only be solved using simulation techniques that cross several engineering disciplines.

A multi-physics environment is defined by coupled processes or systems involving more than one simultaneously occurring physical fields or phenomena. Such an environment is typically described by multiple partial differential equations (PDEs), and tightly coupled such that solving them presents significant challenges with nonlinearities and time-stepping. In general, the more interacting physics in a simulation, the more costly the computation.

Example: almost all practical problems in fluid dynamics involve the interaction between a gas and/or liquid with a solid object, and include a range of associated physics including heat transfer, particle transport, erosion, deposition, flow-induced-stress, combustion and chemical reaction.

Multi-Scale

Cascades-of-scales involve more than two scales with long-range spatio-temporal interactions.

Multi-scale modeling can enable researchers to probe relevant phenomena at smaller scales and seamlessly embed the relevant mechanisms at larger scales to predict the emergent dynamics of the overall system.

In practice, multi-scale modeling is burdened by computational inefficiency, especially with increasing complexity and scales

Example: In the context of biological and behavioural sciences, for instance, multi-scale modeling applications range from the molecular, cellular, tissue, and organ levels all the way to the population level.

Multi-Fidelity

In real-world settings **data** come **from various sources**.

Multi-fidelity modeling aims to synergistically combine abundant, inexpensive, low-fidelity data and sparse, expensive, high-fidelity data from experiments and simulations. Multi-fidelity modeling is often useful in building efficient and robust surrogate models.

- **Physics-Informed ML** (unidirectional): physics providing constraints or other information to direct ML methods.

introducing appropriate observational, inductive, or learning biases that can steer or constrain the learning process to physically consistent solutions

- **Physics-Infused ML** (bidirectional): broader class of methods that can synergise the two computational fields.

For instance a physics-based simulator with an unknown term

$$\frac{dx}{dt} = f_{\text{physics}}(x) + \boxed{g(x)}$$

Neural network $g_{\theta}(x)$

physics provides structure and constraints to the learning process, while ML simultaneously **learns a missing physical component from the data**, which is then incorporated back into the simulator.

Gaussian Processes for PIML

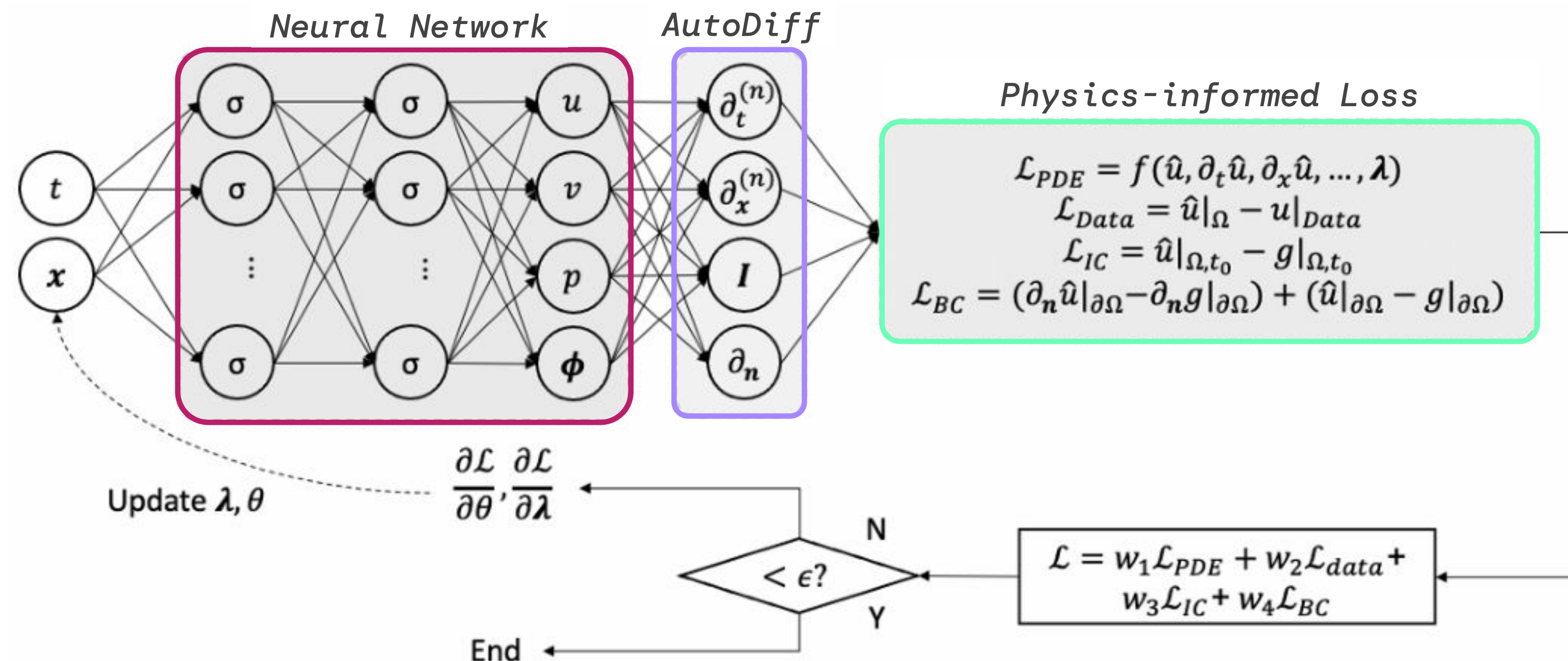
GPs present a useful approach for capturing the physics of dynamical systems. A GP is a Bayesian nonparametric machine learning technique that

- provides a flexible prior distribution over functions,
- enjoys analytical tractability,
- defines kernels for encoding domain structure,
- has a fully probabilistic workflow for principled uncertainty reasoning.

Encoding physics into GP models:

- **Numerical GPs:** covariance functions resulting from temporal discretization of time-dependent partial differential equations (PDEs) which describe the physics
- **Physics-informed basis function GP:** derives a GP kernel directly from the physical model

Neural Networks for PIML

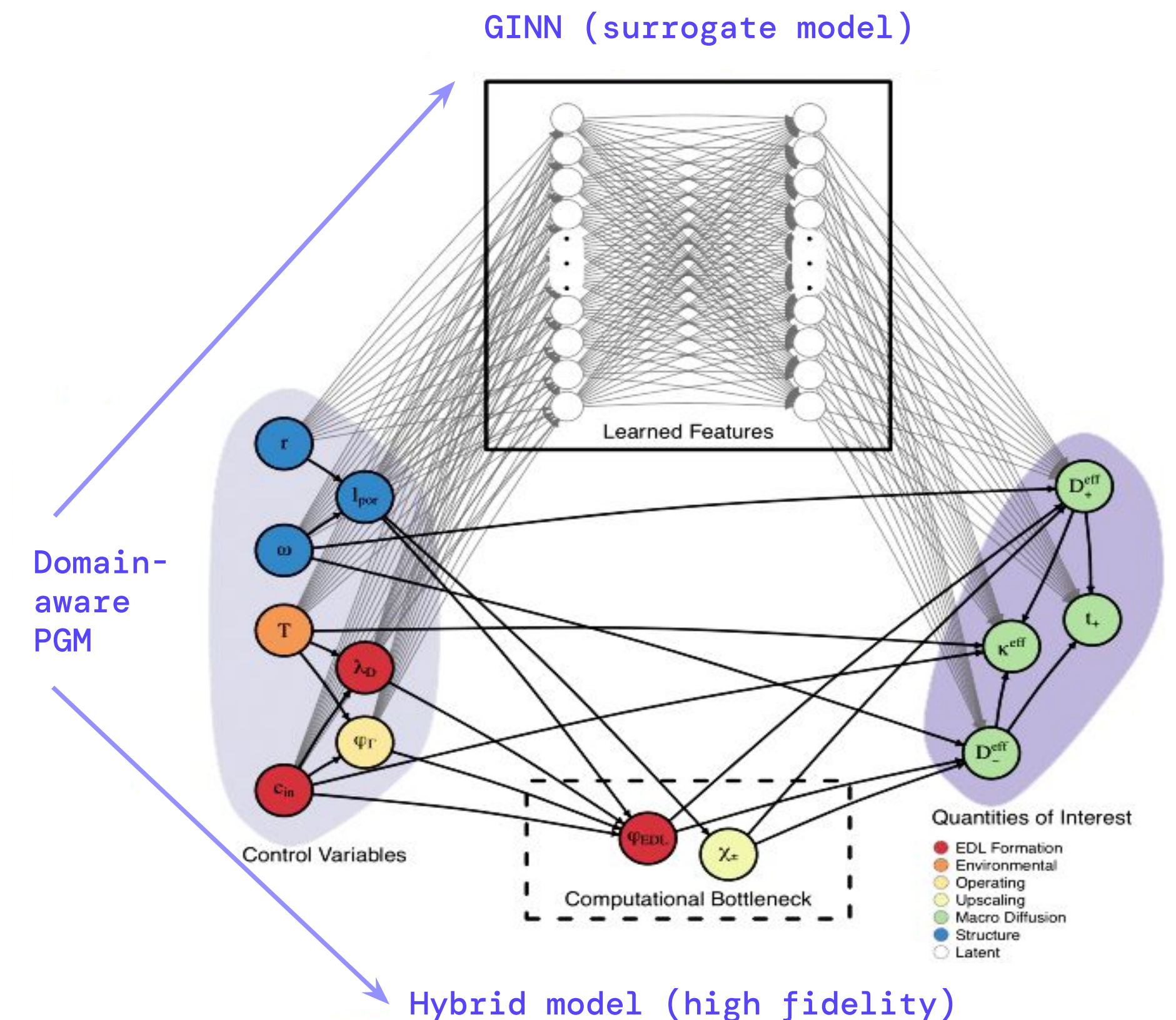


Physics-Informed Neural Networks (PINN): deep neural nets for building surrogates of physics-based models described by partial differential equations (PDEs).

PINN approaches are currently limited to tasks that are characterized by relatively simple and well-defined physics, and often require domain-expert craftsmanship. Even so, the characteristics of many physical systems are often poorly understood or hard to implicitly encode in a neural network architecture.

Neural Networks for PIML

Graph-Informed Neural Networks (GINN): use Probabilistic Graphical Models (PGM) for encoding a priori structure, such as the dependencies among model variables in order to maintain physically sound distributions.



Semi-mechinistic modeling

It follows from the Universal Approximation Theorem that an NN can learn to approximate any sufficiently regular differential equation. The approach of recent **neural-ODE** methods [1] is to learn to approximate differential equations directly from data, but these can perform poorly when required to extrapolate [2].

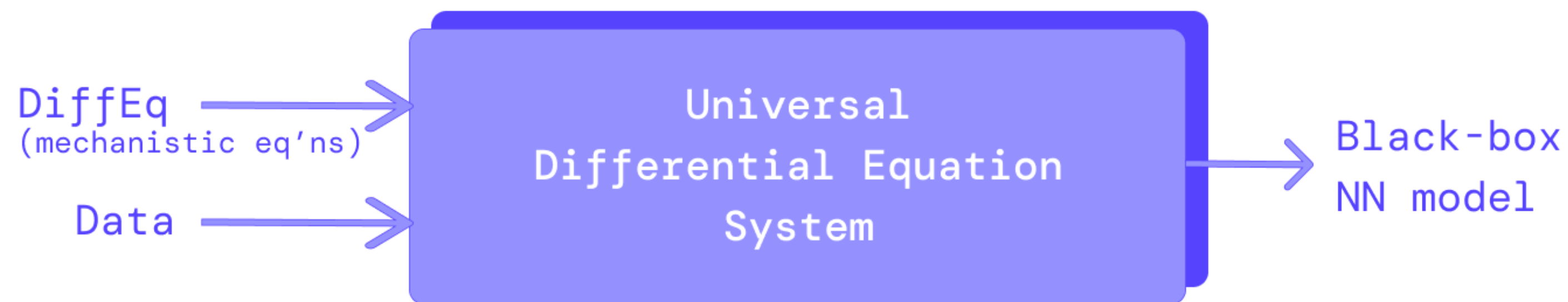
[1] T. Chen, Yulia 2, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

[2] Hanshu Yan, Jiawei Du, V. Tan, and Jiashi Feng. On robustness of neural ordinary differential equations. *ArXiv*, abs/1910.05513, 2020.

Semi-mechinistic modeling

More encouraging for emulation and scientific modeling is to directly utilize mechanistic modeling simultaneously with NNs (or more generally, universal approximator models) in order to allow for arbitrary data-driven model extensions.

Universal Differential Equations (UDE) [3] where part of the differential equation contains a universal approximator model (generally NN, but other options include GP, Chebyshev expansion, or random forest).



[3] Christopher Rackauckas, Y. Ma, Julius Martensen, Collin Warner, K. Zubov, R. Supekar, Dominic J. Skinner, and Ali Ramadhan. Universal differential equations for scientific machine learning. *ArXiv*, abs/2001.04385, 2020.

Uncertainty Quantification

Surrogate models approximate complex physical simulators in order to reduce computational cost. However, their predictions are affected by uncertainties originating from both the **physical system** and the **learning process**.

Uncertainty Quantification (UQ) aims to characterize how these uncertainties propagate to the predictions of the surrogate model.

Two main sources of uncertainty are typically considered:

- **Aleatoric uncertainty**

Uncertainty inherent to the physical system or observations, such as measurement noise, process noise, stochastic forcing, or random physical parameters.

This uncertainty cannot generally be eliminated by collecting more data.

- **Epistemic uncertainty**

Uncertainty due to incomplete knowledge of the system or limited training data. In surrogate modeling, it reflects uncertainty about the learned approximation and is typically larger in poorly sampled or out-of-distribution regions. It can often be reduced by collecting additional informative data or simulations.

UQ should account for uncertainty coming from:

physical system data surrogate approximation physical model

The key point is that a surrogate should not only predict, but also tell us when its prediction is **reliable**.

Simulation-based inference (SBI)

Common challenge: how to link simulation-based models with empirical data.

Numerical simulators typically have parameters whose exact values have to be inferred from data (not known a priori).

SBI designs statistical inference procedures that can be applied to complex simulators.

Stochastic simulators model randomness or uncertainty. **Statistical inference** identifies parameter values consistent with observed data and prior assumptions.

A key ingredient is the likelihood $p(x | \theta)$ of x given parameters θ .

Simulation-based inference (SBI)

- **Bayesian inference:** characterises parameters which are compatible both with data and prior by posterior distribution $p(\theta | x)$, which is proportional to the product of likelihood and prior, $p(\theta | x) \propto p(x | \theta)p(\theta)$
- **Frequentist inference:** construct confidence regions based on hypothesis tests, often using likelihood ratio as test statistic.

If likelihoods can **not** be evaluated, most conventional inference approaches can not be used.

The goal of simulation-based inference is to make statistical inference possible for so-called ***implicit* models** which **allow generating simulated data, but not evaluation of likelihoods**.

Essentials of Machine Learning

1. Supervised Learning

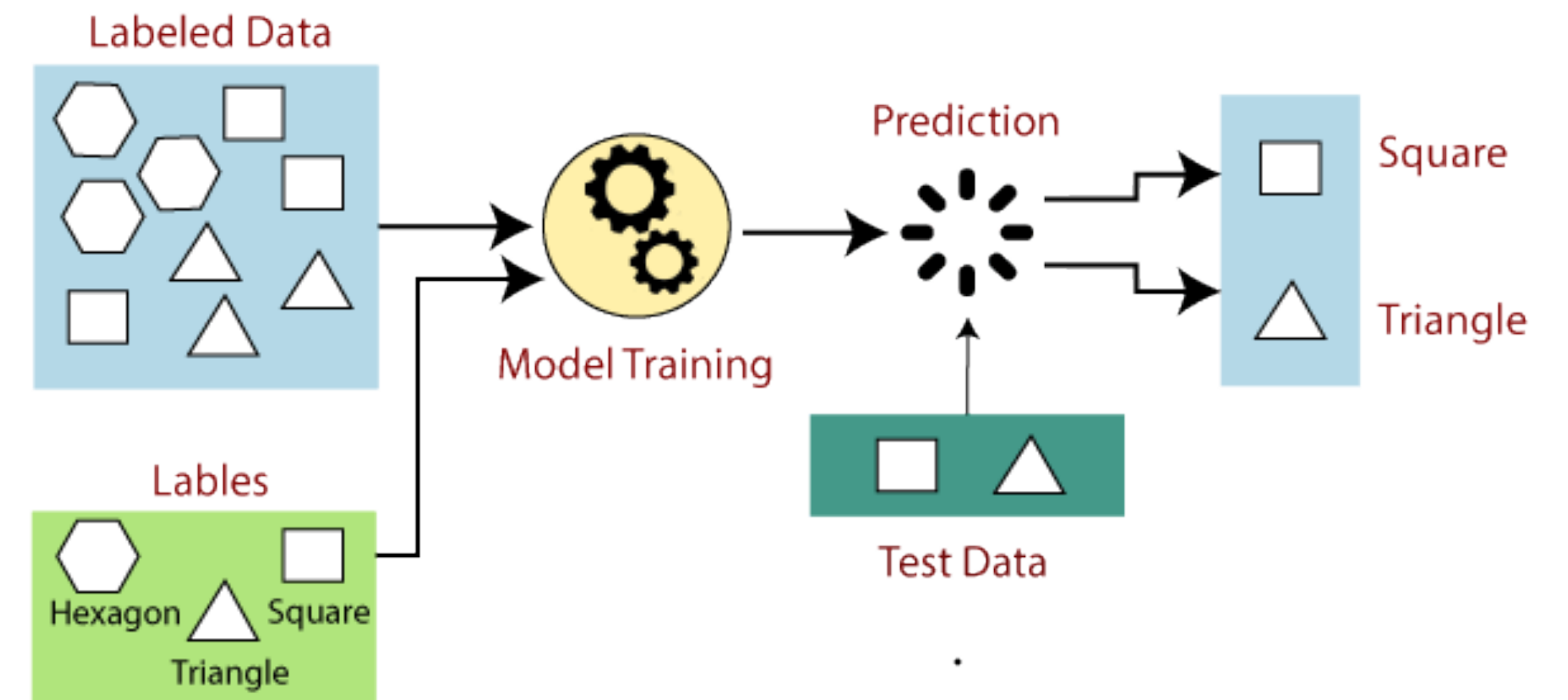
Goal: Learn from labeled data to make predictions.

Data: Each example has an input-output pair (e.g., image labeled as “cat” or “dog”).

Example: Classifying emails as spam or not spam.

Use Case: When lots of labeled data is available.

Fully labeled data, used for prediction.



Essentials of Machine Learning

2. Unsupervised Learning

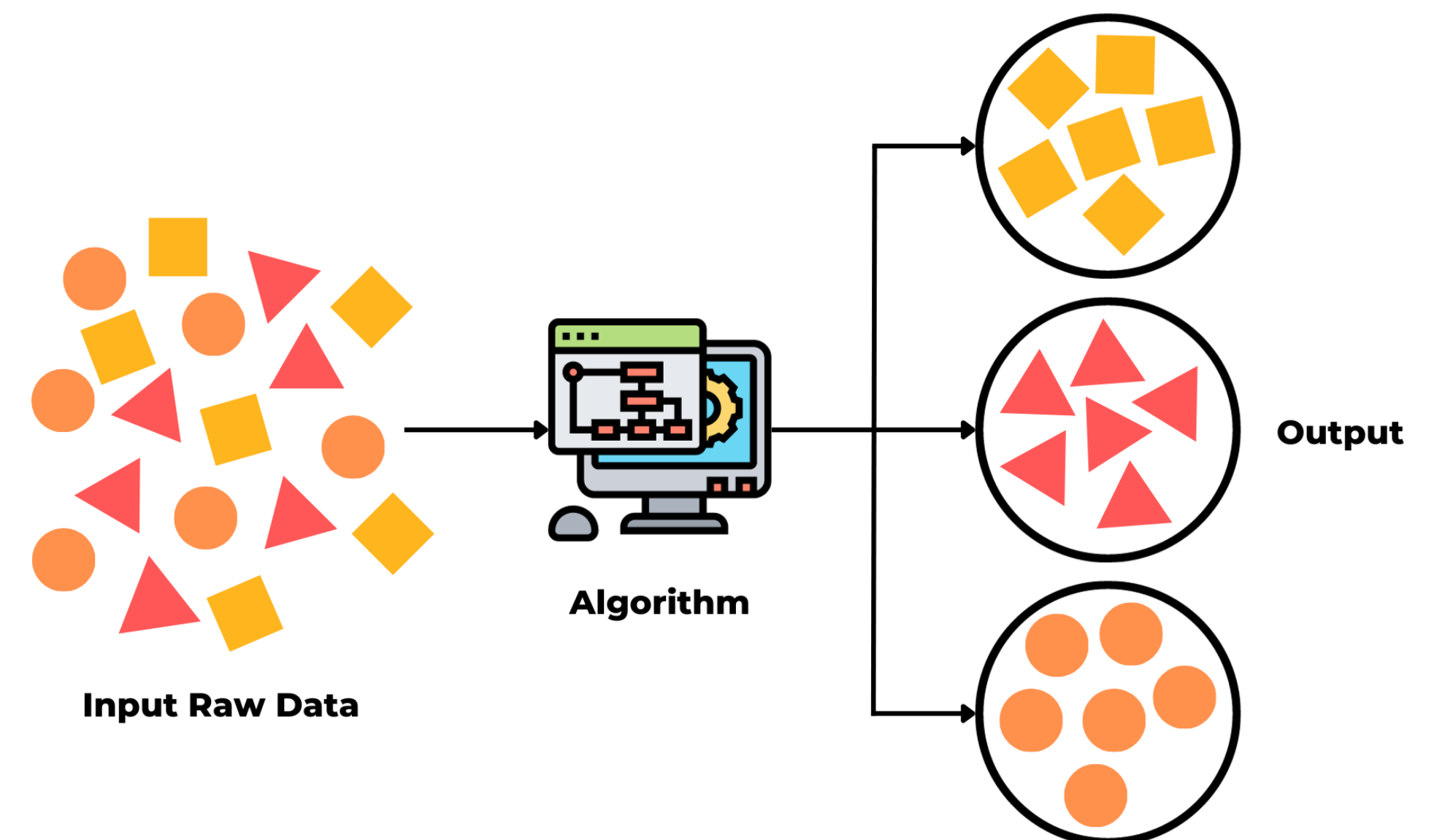
Goal: Find patterns or groupings in unlabeled data.

Data: Only inputs, no labels (e.g., just images without descriptions).

Example: Grouping customers into clusters based on purchasing habits.

Use Case: Useful for understanding the structure of the data

No labels, used for discovering patterns.



Essentials of Machine Learning

3. Semi-Supervised Learning

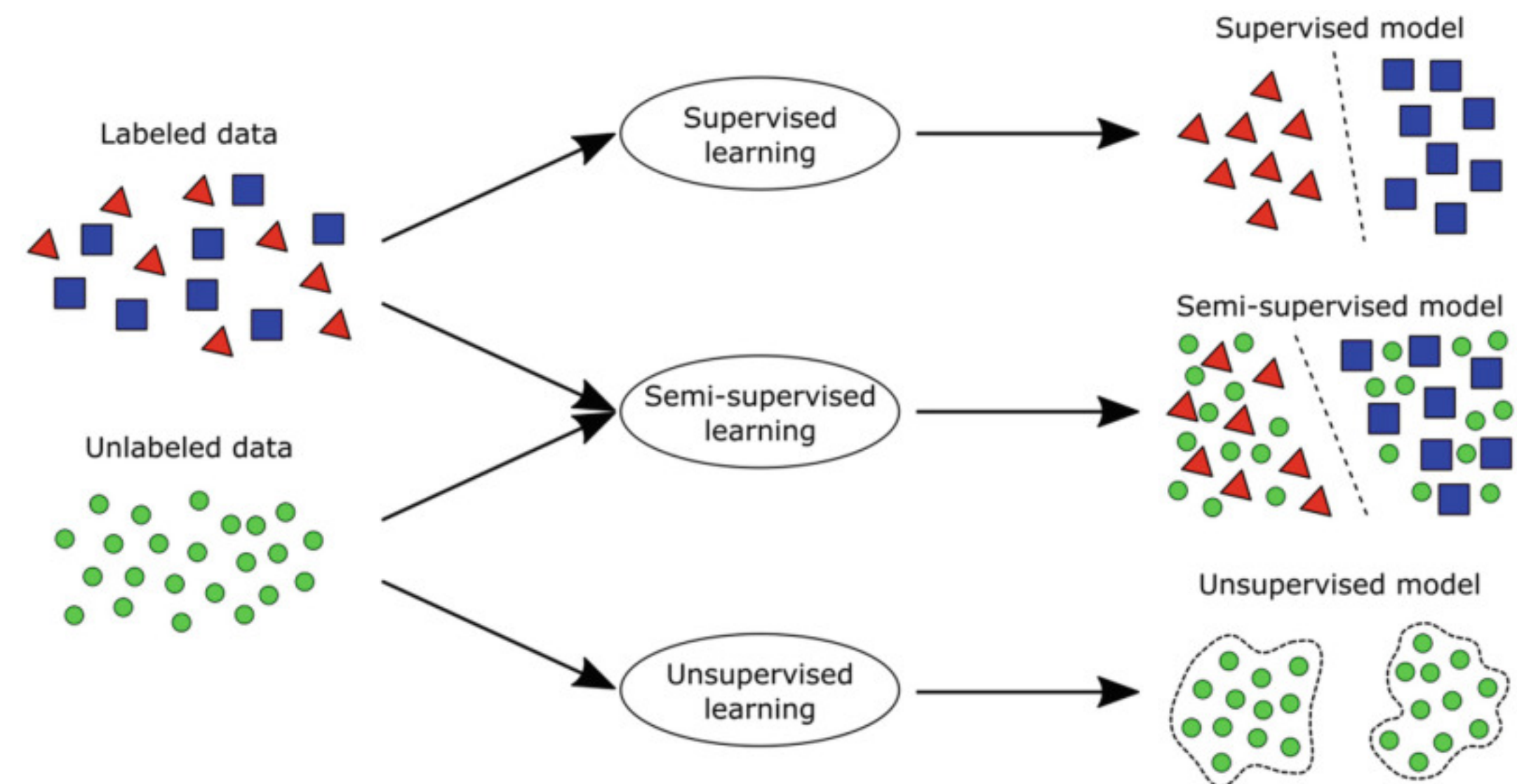
Goal: Use a small amount of labeled data along with a large amount of unlabeled data.

Data: Mix of labeled and unlabeled data (e.g., a few labeled images of animals with many unlabeled ones).

Example: Classifying documents with a few labeled topics.

Use Case: When labeling data is expensive or time-consuming.

Mix of labeled and unlabeled data, balances cost and accuracy.



Essentials of Machine Learning

4. Self-Supervised Learning

Goal: Learn meaningful patterns and representations from unlabeled data by setting up its own learning tasks.

Data: Unlabeled data, with the model creating pseudo-labels or tasks based on the data itself.

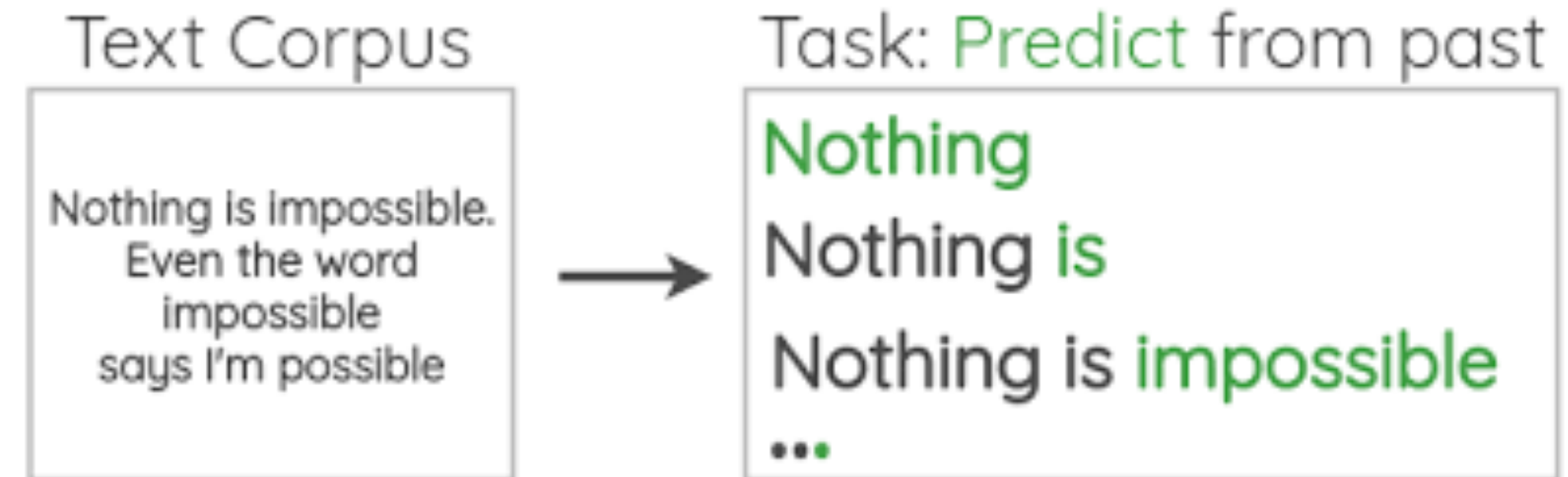
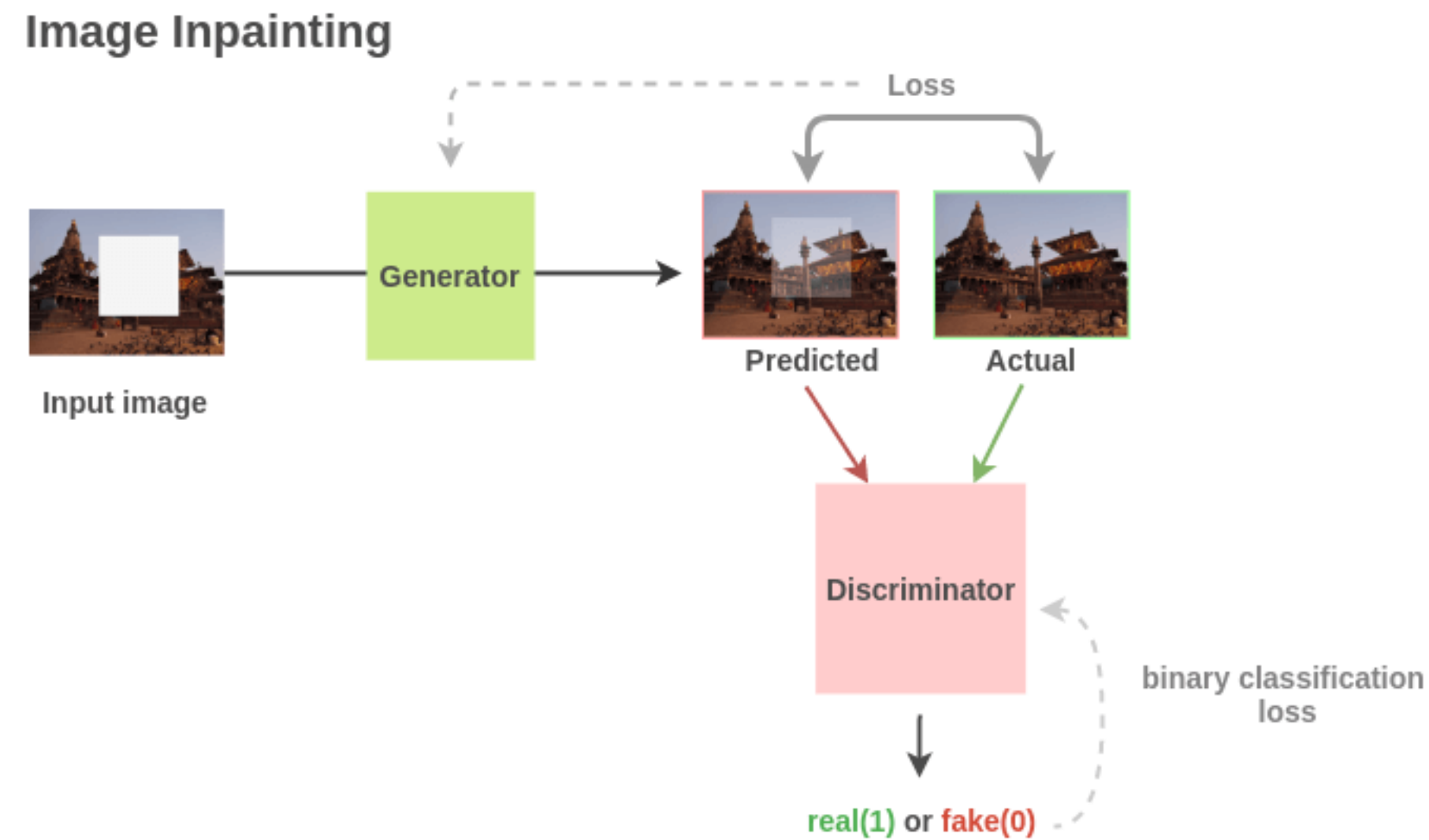
Example: In image inpainting, the model is given an image with a masked section and tasked with predicting the missing content. The model minimizes the difference between its prediction and the actual content in the masked area, using a loss function like mean squared error (MSE).

Use Case: Useful in domains with abundant unlabeled data but limited labeled data, such as natural language processing (e.g., predicting missing words in text) or computer vision (e.g., filling in missing parts of images).

Self-supervised learning is often used to generate useful representations of data without relying on labeled examples (**representation learning**).

Essentials of Machine Learning

4. Self-Supervised Learning



What is Supervised Learning?

Goal: Predict an output (target) based on input data.

How It Works:

The model learns from labeled examples (input-output pairs).

Key Components:

Dataset: Collection of data points with inputs and labeled outputs.

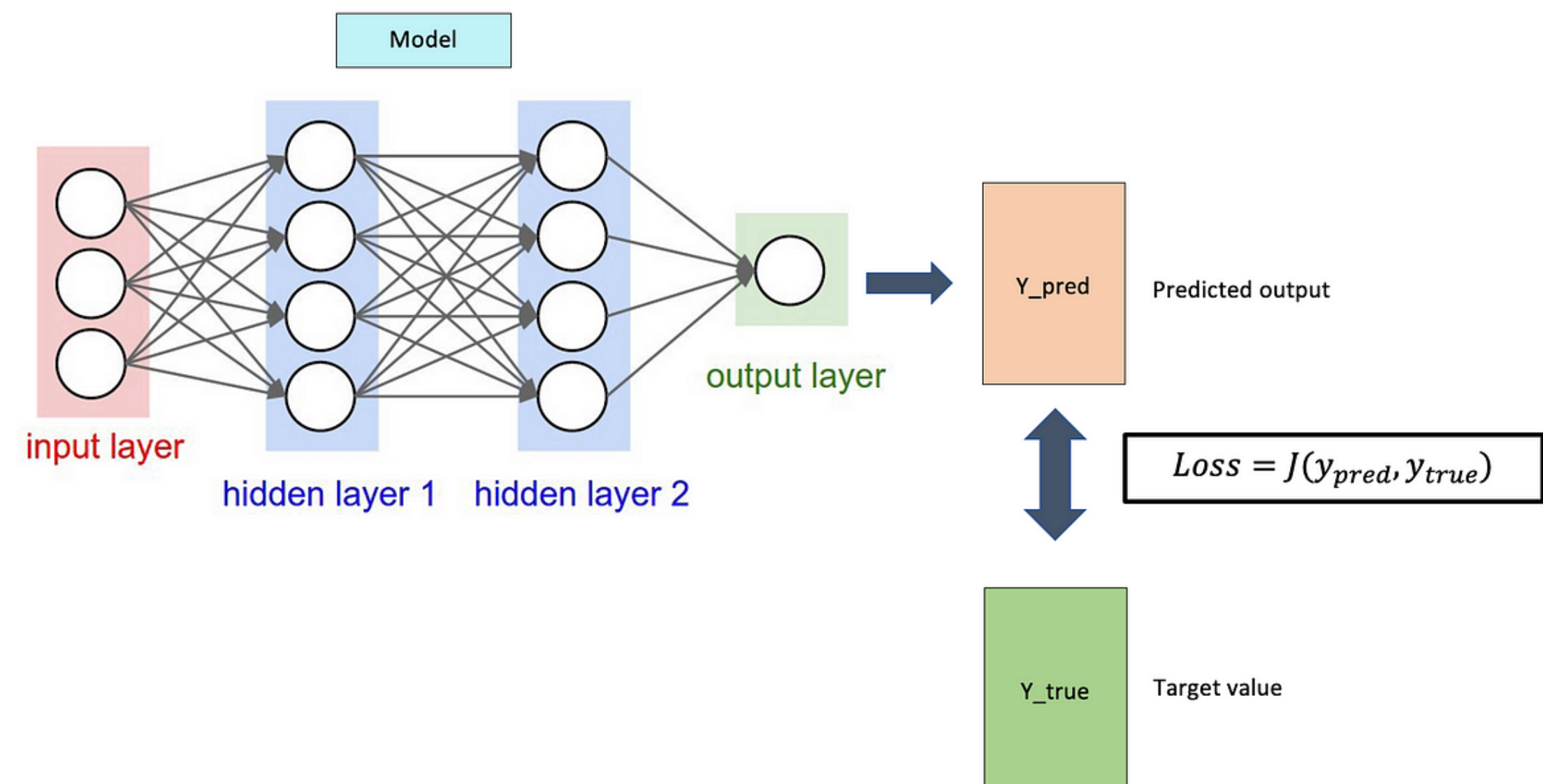
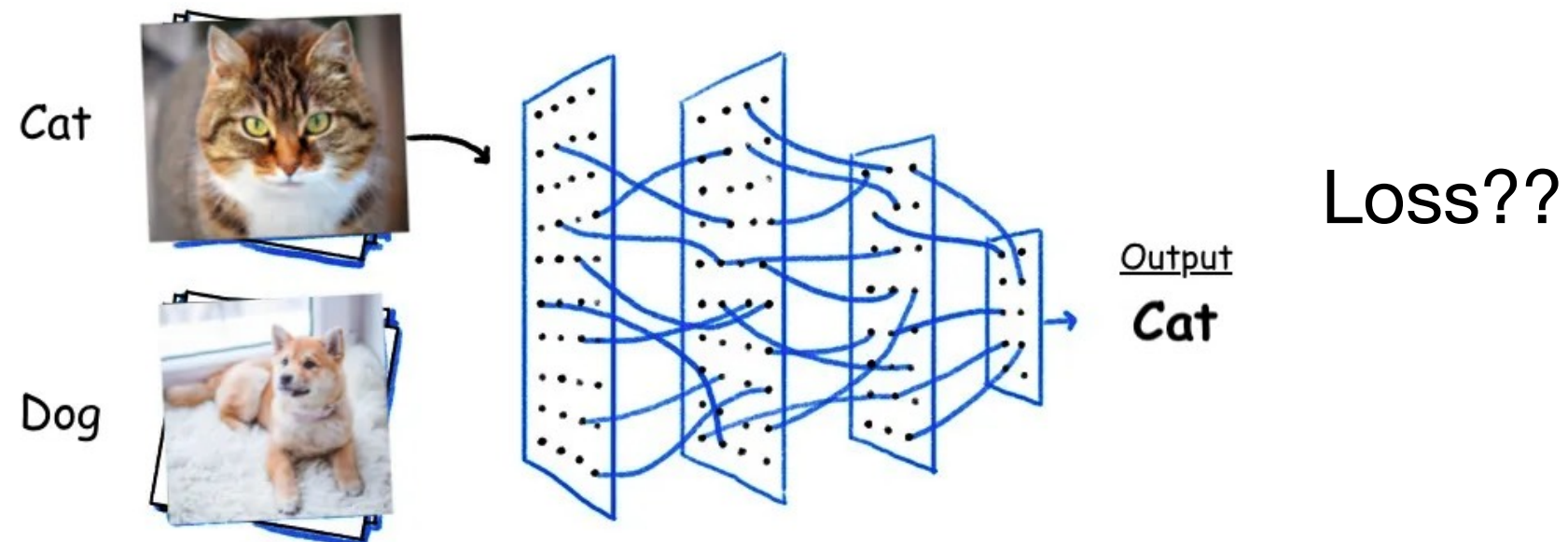
Model: A mathematical structure (e.g., linear regression, neural network) that tries to learn patterns in data.

Loss Function: Measures how far off the model's predictions are from the actual answers.

Examples:

Image classification (e.g., cat vs. dog).

Predicting house prices based on size, location, etc.



Gradient-based Optimization

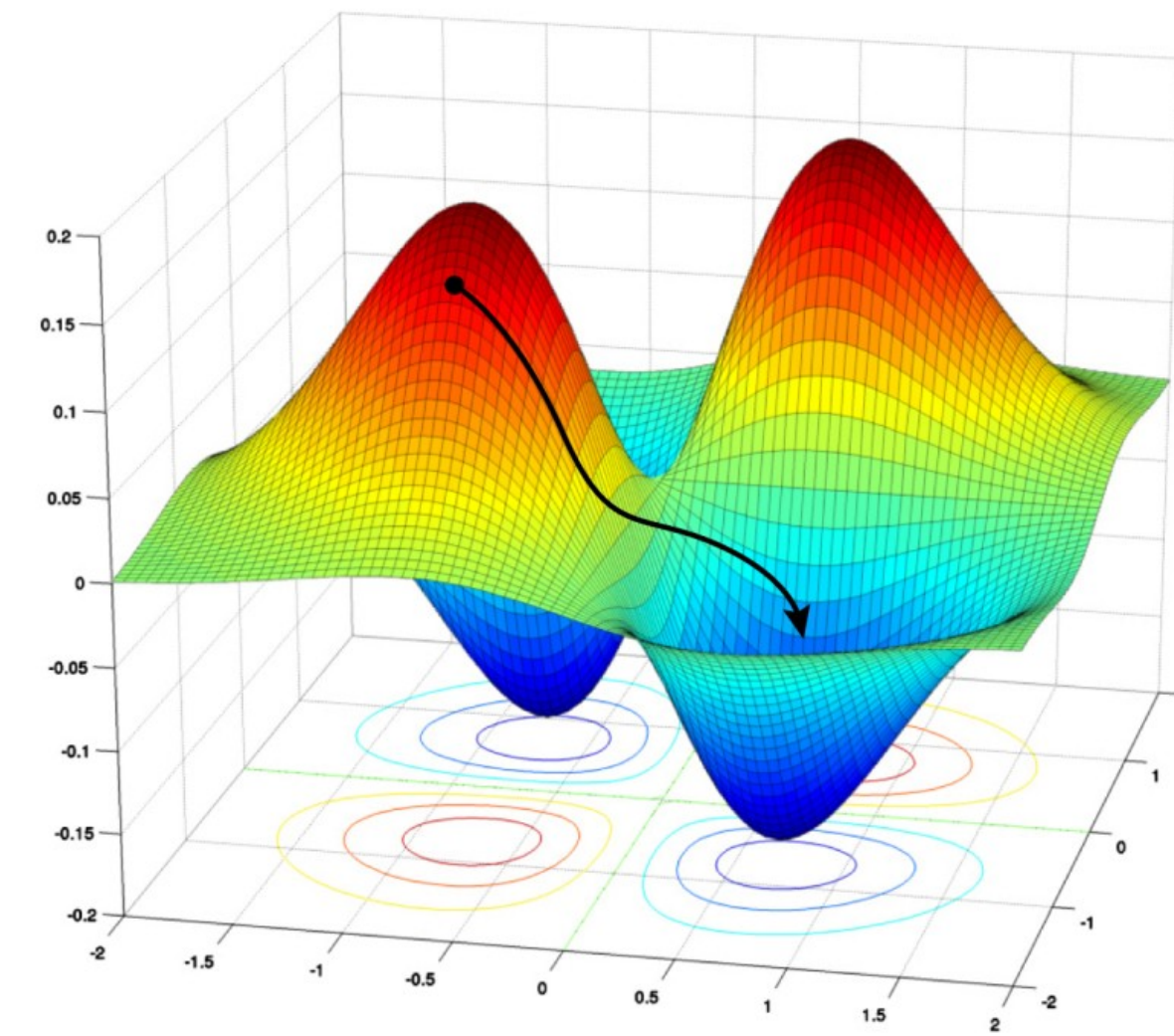
Gradient Descent is a method to minimize the **loss function** by adjusting the model's parameters.

Goal: Find the values that make the predictions as close as possible to the actual answers.

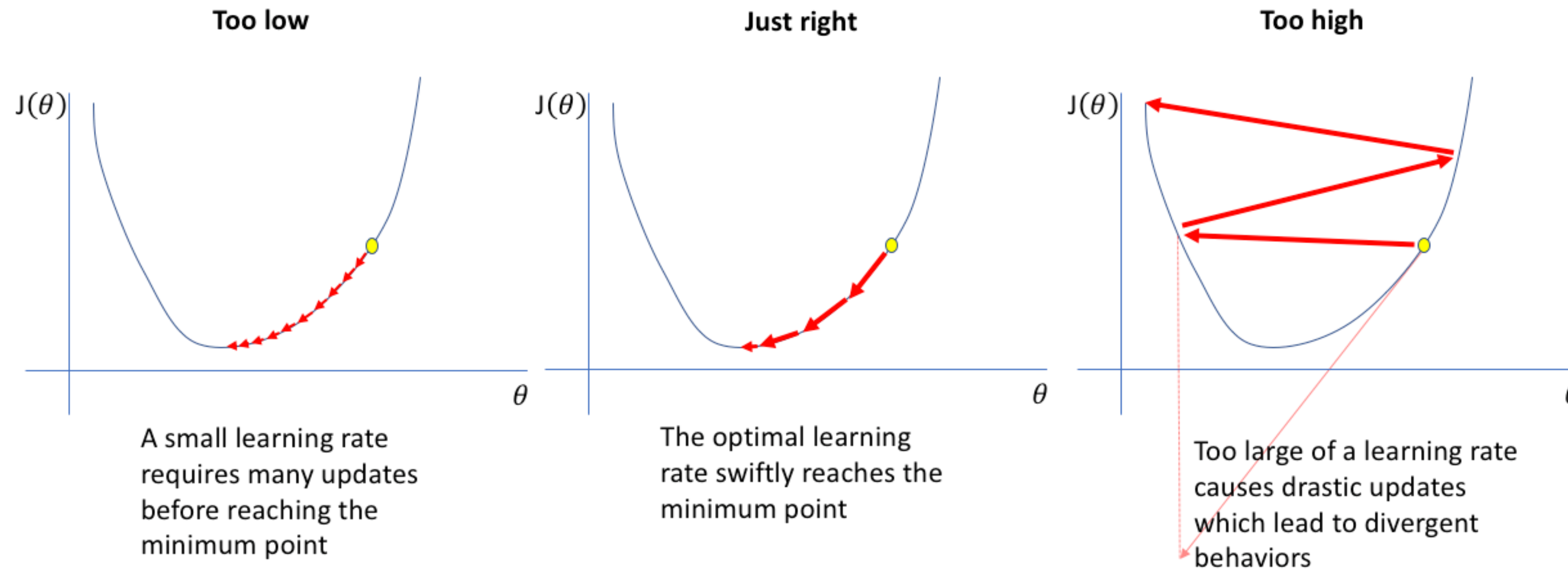
Visualization: Imagine finding the lowest point in a valley – Gradient Descent helps us step down the slope

Algorithm:

- Calculate the **gradient** of the loss function – the direction in which the loss increases.
- Take steps in the **opposite direction** (downhill) to reduce the loss.
- Repeat until the model is optimized or the loss is low enough.
- **Learning Rate:** The size of each step. Too large → might overshoot; too small → slow progress.



Gradient-based Optimization



Why Gradient Descent is Important?

- **Efficiency:** Helps find the best model parameters without testing every possibility.
- **Scalability:** Essential for large datasets and complex models.
- **Applications:** Used widely in ML for tasks like regression, classification, neural networks, etc

Coming next...

Automatic Differentiation/Differentiable Programming